

Inside Claude Code: An Architecture Deep Dive

Created on April 01, 2026

📅 2026 · #llms #ai-agents #developer-tools · 💡 technical-blog

- Claude Code v2.1.88 — Architecture Deep Dive
 - Table of Contents
 - 1. High-Level Architecture Diagram
 - Narrative
 - 2. Directory Map
 - 3. Startup & Bootstrap Sequence Diagram
 - Narrative
 - 4. The Query Loop — Core Interaction Flow
 - Narrative
 - 5. Tool Execution Sequence Diagram
 - Narrative
 - 6. Dependency & Layer Diagram
 - Narrative
 - 7. Key Abstractions & Data Models
 - The Tool Interface (src/Tool.ts)
 - AppState (src/state/AppStateStore.ts)
 - Message Types
 - Permission Decision Flow
 - 8. Communication Patterns
 - How Components Communicate
 - Inter-Agent Communication (Swarms)
 - 9. Narrative Walkthrough: A Message End-to-End
 - Step 1: Input Capture (Presentation Layer)
 - Step 2: Message Creation (Application Layer)
 - Step 3: Context Gathering (Application Layer)
 - Step 4: API Call (Infrastructure Layer)
 - Step 5: Streaming Response (Application → Presentation)
 - Step 6: Tool Calls (Application → Domain → Infrastructure)
 - Step 7: Loop Back (Application Layer)
 - Step 8: Completion (Application → Presentation)
 - Step 9: Persistence (Infrastructure Layer)
 - Why Is It Structured This Way?
 - 10. Deep Dive: The Grep Tool (Agentic Search)
 - Architecture
 - Input Schema
 - Execution Flow
 - Ripgrep Binary Resolution (src/utils/ripgrep.ts)
 - Error Recovery
 - Pagination System
 - Result Formatting
 - Permission & Security
 - Tool Properties
 - 11. Deep Dive: Memory Management & Context Compaction
 - The Problem
 - Architecture Overview
 - Key Files

- [How Tokens Are Counted](#)
- [Layer 1: Tool Result Budget](#)
- [Layer 2: Snip Compaction \(feature-gated: HISTORY_SNIP\)](#)
- [Layer 3: Microcompaction \(per-turn\)](#)
- [Layer 4: Context Collapse \(feature-gated: CONTEXT_COLLAPSE, internal\)](#)
- [Layer 5: Autocompact \(threshold-based\)](#)
 - [Strategy A: Session Memory Compaction \(SM-Compact\)](#)
 - [Strategy B: Full Conversation Compaction \(fallback\)](#)
- [Background: Session Memory Extraction](#)
- [Post-Compact Cleanup](#)
- [Integration in the Query Loop \(src/query.ts\)](#)
- [Reactive vs. Proactive Compaction](#)
- [Lifecycle of a Long Conversation](#)
- [Key Constants](#)

This is a live document as I explore the Claude Code leaked repo — this is mainly LLM generated as I navigate the repo and study how it implements things like memory, compaction, agentic search, etc.

Claude Code v2.1.88 — Architecture Deep Dive

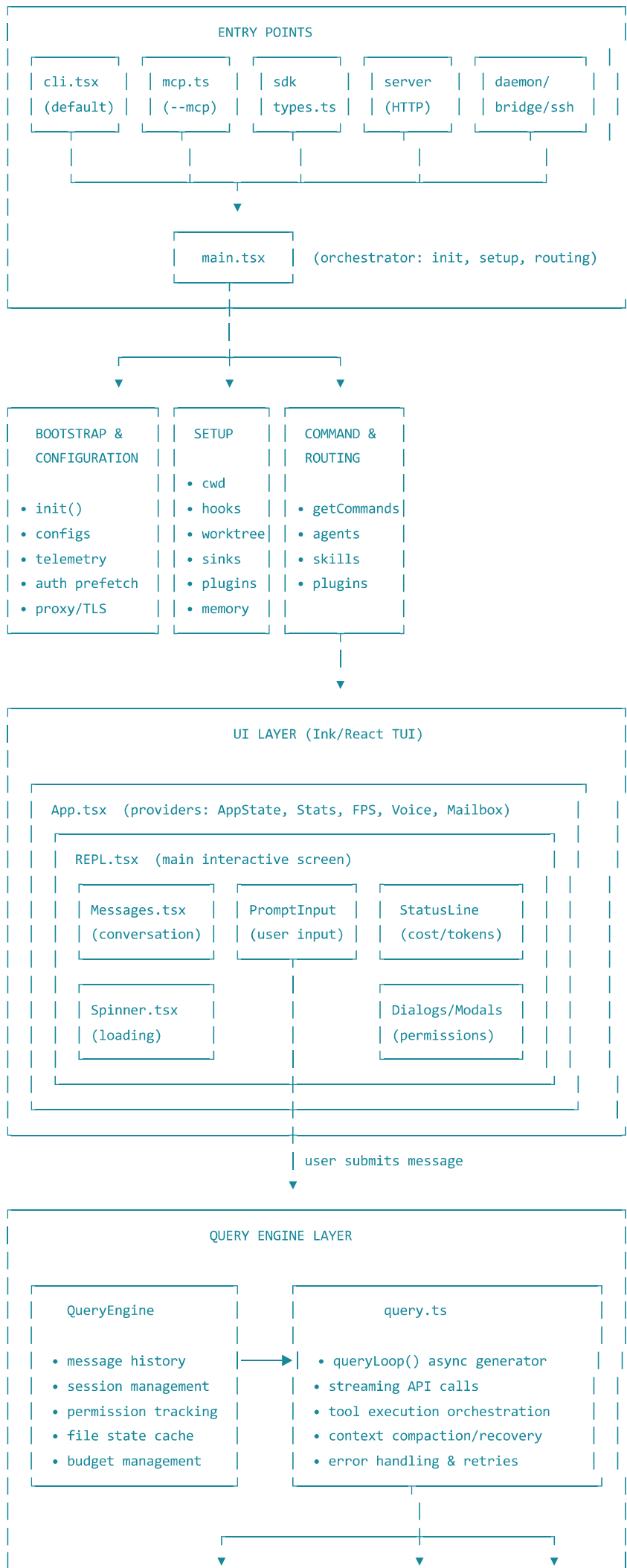
A comprehensive visual and narrative guide to how Claude Code works internally. Written for someone new to the codebase.

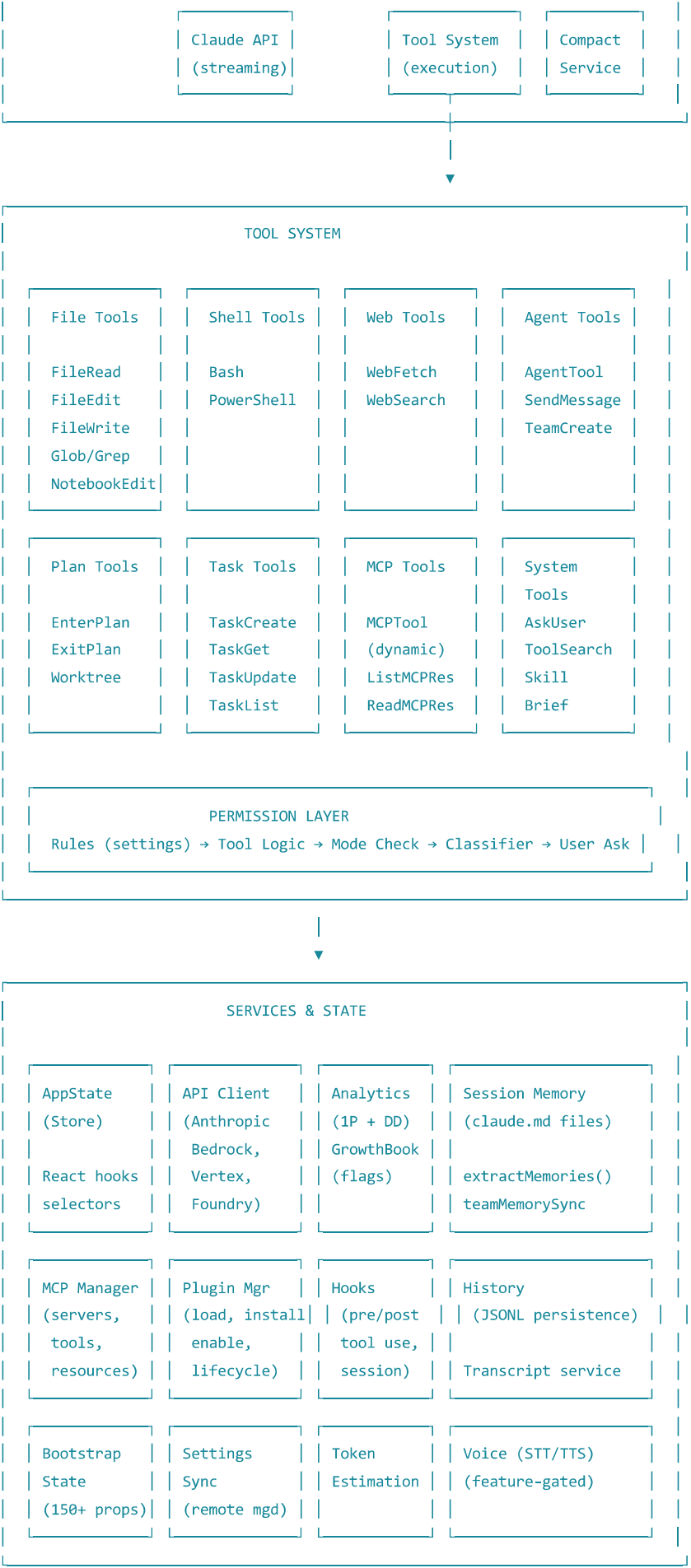
Table of Contents

- [1. High-Level Architecture Diagram](#)
- [2. Directory Map](#)
- [3. Startup & Bootstrap Sequence Diagram](#)
- [4. The Query Loop — Core Interaction Flow](#)
- [5. Tool Execution Sequence Diagram](#)
- [6. Dependency & Layer Diagram](#)
- [7. Key Abstractions & Data Models](#)
- [8. Communication Patterns](#)
- [9. Narrative Walkthrough: A Message End-to-End](#)
- [10. Deep Dive: The Grep Tool \(Agentic Search\)](#)
- [11. Deep Dive: Memory Management & Context Compaction](#)

1. High-Level Architecture Diagram

This is the 30,000-foot view. Every box is a module group; every arrow is a dependency or data flow direction.





Narrative

Claude Code is a **terminal-based AI coding assistant** built as a TypeScript application with a custom React/Ink TUI (Terminal User Interface). The architecture follows a layered pattern:

1. **Entry Points** route to the right mode (CLI, MCP server, SDK, daemon, etc.)
2. **Bootstrap** loads configuration, telemetry, auth, and networking
3. **Setup** prepares the working directory, hooks, plugins, and file watchers
4. **UI Layer** renders the terminal interface using a custom Ink/React renderer
5. **QueryEngine** orchestrates the conversation: sending messages to Claude, streaming responses, and executing tools
6. **Tool System** provides 50+ tools the model can invoke, each with its own permission logic
7. **Services & State** underpin everything: API clients, analytics, MCP servers, memory, history

The key insight is that **the QueryEngine is the heart** — it sits between the UI and the Claude API, managing the agentic loop of “send message → get response → execute tools → send results → repeat.”

2. Directory Map

claude-code-source-code/

```
|
├─ src/                                # All source code (1,884 files, 512K LOC)
|   |
|   ├─ entrypoints/                    # [ENTRY] CLI, MCP, SDK entry points
|   |   ├─ cli.tsx                     # Primary CLI entry – fast-path routing
|   |   ├─ init.ts                     # Lazy config/telemetry initialization
|   |   ├─ mcp.ts                      # MCP server mode (--mcp flag)
|   |   └─ agentSdkTypes.ts            # SDK type exports
|   |
|   ├─ main.tsx                       # [ORCHESTRATOR] Full CLI init, permission, REPL launch
|   ├─ setup.ts                       # [SETUP] CWD, hooks, worktree, sinks, memory
|   ├─ replLauncher.tsx               # [LAUNCH] Final step: renders <App><REPL/></App>
|   |
|   ├─ QueryEngine.ts                 # [CORE] Conversation manager – message history, sessions
|   ├─ query.ts                       # [CORE] The query loop – API calls, tool orchestration
|   ├─ query/                          # [CORE] Query subsystems
|   |   ├─ config.ts                  # Query configuration builder
|   |   ├─ deps.ts                   # Dependency injection (API, compact, etc.)
|   |   ├─ tokenBudget.ts             # Token budget tracking & decisions
|   |   └─ stopHooks.ts               # Lifecycle hooks at query boundaries
|   |
|   ├─ Tool.ts                        # [TOOLS] Tool interface definition + buildTool()
|   ├─ tools.ts                       # [TOOLS] Tool registry – getAllBaseTools()
|   ├─ tools/                         # [TOOLS] 50+ tool implementations
|   |   ├─ AgentTool/                 # Spawn sub-agents
|   |   ├─ BashTool/                 # Shell command execution
|   |   ├─ FileReadTool/             # File reading (images, PDFs, notebooks)
|   |   ├─ FileEditTool/             # Diff-based file editing
|   |   ├─ FileWriteTool/            # File creation
|   |   ├─ GlobTool/                 # File pattern matching
|   |   ├─ GrepTool/                 # Content search (ripgrep)
|   |   ├─ WebFetchTool/             # HTTP fetching
|   |   ├─ WebSearchTool/            # Web search
|   |   └─ ...                       # 40+ more tools
|   |
|   ├─ commands.ts                   # [COMMANDS] Slash command registry
|   ├─ commands/                     # [COMMANDS] 100+ slash command implementations
|   |
|   ├─ state/                         # [STATE] Application state management
|   |   ├─ AppStateStore.ts          # AppState type + defaults (300+ lines)
|   |   ├─ AppState.tsx              # React hooks: useAppState, useSetAppState
|   |   ├─ store.ts                  # Generic Store<T> implementation
|   |   ├─ selectors.ts              # Pure state selectors
|   |   └─ onChangeAppState.ts       # Side-effect handler for state changes
|   |
|   ├─ bootstrap/                   # [BOOTSTRAP] Global mutable state container
|   |   └─ state.ts                  # 150+ session properties (metrics, auth, UI)
|   |
|   ├─ services/                    # [SERVICES] 18 service modules
|   |   ├─ api/                      # Claude API client (Anthropic/Bedrock/Vertex)
|   |   ├─ analytics/                # Event logging, Datadog, GrowthBook
|   |   ├─ mcp/                      # MCP server management & auth
|   |   ├─ tools/                    # Tool execution & orchestration
|   |   ├─ plugins/                  # Plugin loading & installation
|   |   ├─ compact/                  # Transcript compaction
|   |   ├─ SessionMemory/            # Per-turn context gathering
|   |   ├─ extractMemories/          # Memory extraction from conversations
|   |   └─ voice/                    # Voice mode (STT/TTS)
|   |
|   └─ components/                  # [UI] React/Ink UI components
```

			└─ App.tsx	#	Root provider wrapper		
			└─ Messages.tsx	#	Conversation message list		
			└─ Message.tsx	#	Single message renderer		
			└─ VirtualMessageList	#	Virtualized scrolling		
			└─ PromptInput/	#	Text input area		
			└─ Spinner.tsx	#	Loading indicators		
			└─ StatusLine.tsx	#	Bottom status bar		
			└─ Markdown.tsx	#	Markdown rendering		
			└─ screens/	#	[SCREENS] Top-level screen views		
					└─ REPL.tsx	#	Main interactive screen (895KB!)
					└─ Doctor.tsx	#	Diagnostics
					└─ ResumeConversation	#	Session resumption
			└─ hooks/	#	[REACT HOOKS] Custom React hooks		
			└─ skills/	#	[SKILLS] Bundled skill definitions		
			└─ plugins/	#	[PLUGINS] Built-in plugin definitions		
			└─ context/	#	[CONTEXT] React context providers		
			└─ schemas/	#	[SCHEMAS] Zod validation schemas		
			└─ types/	#	[TYPES] Shared TypeScript types		
			└─ utils/	#	[UTILS] Shared utilities		
					└─ permissions/	#	Permission rule engine
					└─ hooks.ts	#	Hook execution engine
					└─ tasks.ts	#	Task/todo persistence
			└─ ink/	#	[INK] Custom terminal React renderer		
					└─ ink.tsx	#	Core renderer (~9,000 lines)
					└─ reconciler.ts	#	React reconciler
					└─ dom.ts	#	Component tree → terminal output
					└─ layout/	#	Yoga-based layout engine
					└─ events/	#	Keyboard, mouse, focus events
			└─ context.ts	#	[CONTEXT] System + user context builders		
			└─ history.ts	#	[HISTORY] JSONL command history		
			└─ Task.ts	#	[TASKS] Task type definitions		
			└─ tasks.ts	#	[TASKS] Task registration		
			└─ cost-tracker.ts	#	[COST] USD cost tracking		
			└─ stubs/	#	Build stubs for Bun intrinsics		
			└─ scripts/	#	Build scripts (esbuild)		
			└─ tools/	#	Build tooling		
			└─ vendor/	#	Native module stubs		
			└─ types/	#	Additional type declarations		
			└─ docs/	#	Analysis reports (EN/JA/KO/ZH)		

3. Startup & Bootstrap Sequence Diagram

This traces what happens from `node cli.js` to the interactive REPL appearing.

User runs: node cli.js [args]

PHASE 1: FAST-PATH ROUTING (cli.tsx – zero imports for --version)

```
cli.tsx:main()
|
|   └─> --version / -v ? ──> print version, exit (no imports!)
|   └─> --mcp ? ─────────> startMCPServer(), exit
|   └─> --daemon-worker ? ──> daemon fast-path, exit
|   └─> remote-control / ssh ? ─> bridge/remote, exit
|   └─> ps / logs / attach ? ─> session management, exit
|
|   └─> (default) ─────────> import('./main.js') → cliMain()
```

PHASE 2: INITIALIZATION (main.tsx + init.ts)

```
main.tsx:main()
|
|   └─ Set security env vars, SIGINT handler
|   └─ Parse CLI flags (--model, --print, --debug, etc.)
|
|   ▼
init() [memoized – runs once]
|
|   └─ enableConfigs()           ← load ~/.claude/settings.json
|   └─ applySafeConfigEnvironmentVariables()
|   └─ applyExtraCACertsFromConfig()
|   └─ setupGracefulShutdown()
|   └─ configureGlobalAgents()   ← proxy + mTLS setup
|   └─ preconnectAnthropicApi()  ← TCP pre-connect (latency opt)
|
|   └─ [fire-and-forget background]:
|       └─ initialize1PEventLogging()
|       └─ populateOAuthAccountInfo()
|       └─ detectCurrentRepository()
|       └─ initRemoteManagedSettings()
|
|   └─ Show trust dialog if needed
```

PHASE 3: TELEMETRY + PERMISSIONS

```
|
|   └─ initializeTelemetryAfterTrust() ← OpenTelemetry setup
|   └─ getPermissionContext()
|   └─ Show permission dialog (interactive only)
|
|   └─ initializeToolPermissionContext()
|   └─ initBuiltinPlugins()         ← register bundled plugins
|   └─ initBundledSkills()          ← register bundled skills
```

PHASE 4: SETUP (setup.ts – parallel with command loading)

```
setup()
|
```

```

└─ Check Node.js >= 18
└─ setCwd(cwd) ← MUST come before hooks load
└─ captureHooksConfigSnapshot() ← freeze hook config
└─ initializeFileChangedWatcher()
|
└─ [if --worktree]:
|   └─ createWorktreeForSession()
|       └─ process.chdir(worktreePath)
|
└─ initSessionMemory()
└─ lockCurrentVersion()
|
└─ [fire-and-forget]:
|   └─ getCommands() ← scan for slash commands
|   └─ loadPluginHooks()
|   └─ setupPluginHookHotReload()
|   └─ registerAttributionHooks()
|       └─ prefetchApiKeyIfSafe()
|
└─ initSinks() ← error log + analytics
└─ logEvent('tengu_started') ← session start beacon

```

```

| PHASE 5: COMMAND & AGENT LOADING (parallel with setup) |

```

```

await Promise.all([
  getCommands(cwd), ← all slash commands
  getAgentDefinitionsWithOverrides(cwd) ← agent registry
])

```

```

| PHASE 6: REPL LAUNCH |

```

```

launchRepl(root, appProps, replProps, renderAndRun)
|
└─ import('./components/App.js')
└─ import('./screens/REPL.js')
└─ renderAndRun(root, <App><REPL /></App>)
    |
    └─ Ink TUI event loop starts ← user sees the prompt

```

Narrative

The startup is designed for **speed**. Phase 1 handles trivial commands (`--version`) with zero module imports. Only the default interactive path loads the heavy `main.tsx` . Initialization is memoized and parallelized — network pre-connection, auth prefetching, and repository detection all happen concurrently. Setup and command loading run in parallel via `Promise.all` . The REPL is the final step: it dynamically imports the React components and starts the Ink terminal renderer.

4. The Query Loop — Core Interaction Flow

This is the most important flow in the entire codebase. It's what happens every time you type a message.

QUERY LOOP (query.ts → queryLoop())

This is an async generator that yields streaming events back to the UI

User types message



```
QueryEngine
.submitMessage()

• Wraps canUseTool
• Processes input
• Updates AppState
• Records transcript
```



PREPARE

1. Build system prompt (base + context + tools + skills)
2. Gather user context (claude.md files, git status, date)
3. Normalize message history for API
4. Apply compaction if needed (snip, microcompact, autocompact)



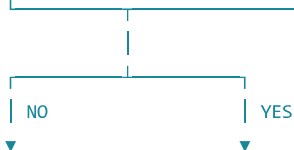
CALL CLAUDE API (streaming)

```
POST /v1/messages {
  model: "claude-opus-4-6-...",
  messages: [...conversation history...],
  system: "You are Claude Code...",
  tools: [...50+ tool schemas...],
  stream: true
}
```

```
for await (const event of stream) {
  yield event → UI renders it
}
```



Response contains tool_use blocks?

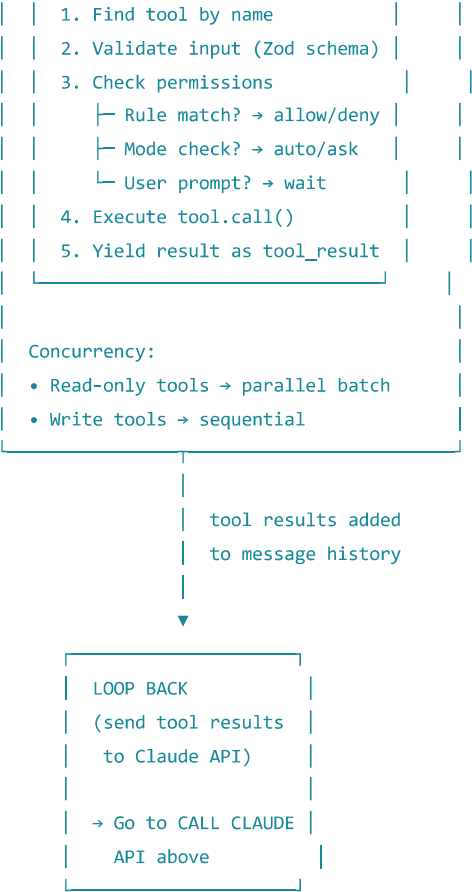


DONE

```
Return Terminal
{ reason:
  'completed' }
```

EXECUTE TOOLS

For each tool_use block:



- RECOVERY PATHS (when things go wrong):
- Context too long → autocompact / reactive compact / context collapse
 - Max output tokens → escalate output limit, retry
 - Model error → fallback model retry
 - User interrupt → abort controller, yield partial results
 - Budget exceeded → stop with 'max_turns' or token budget reason

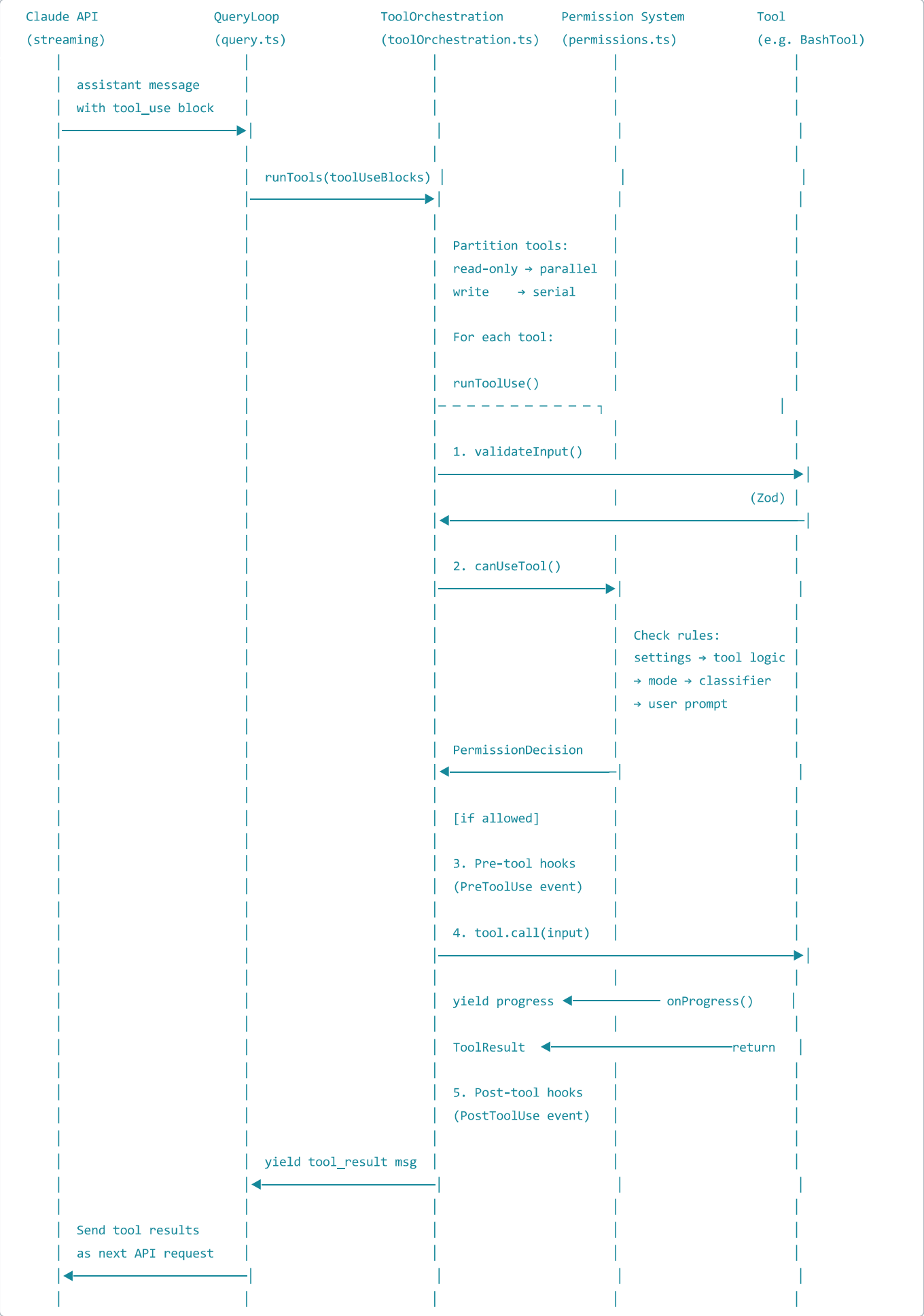
Narrative

The query loop is an **async generator** — it **yield**s streaming events as they arrive from the Claude API, allowing the UI to render responses incrementally. When Claude's response includes **tool_use** blocks (e.g., "read this file" or "run this command"), the loop pauses streaming, executes the tools (with permission checks), and then loops back to send the tool results to Claude. This continues until Claude responds with just text (no tool calls), at which point the loop terminates.

The loop also handles recovery: if the context gets too long, it compacts the conversation history. If the model hits output limits, it escalates. If the user interrupts, it aborts gracefully.

5. Tool Execution Sequence Diagram

This traces a single tool call from Claude's response to the result being sent back.



Narrative

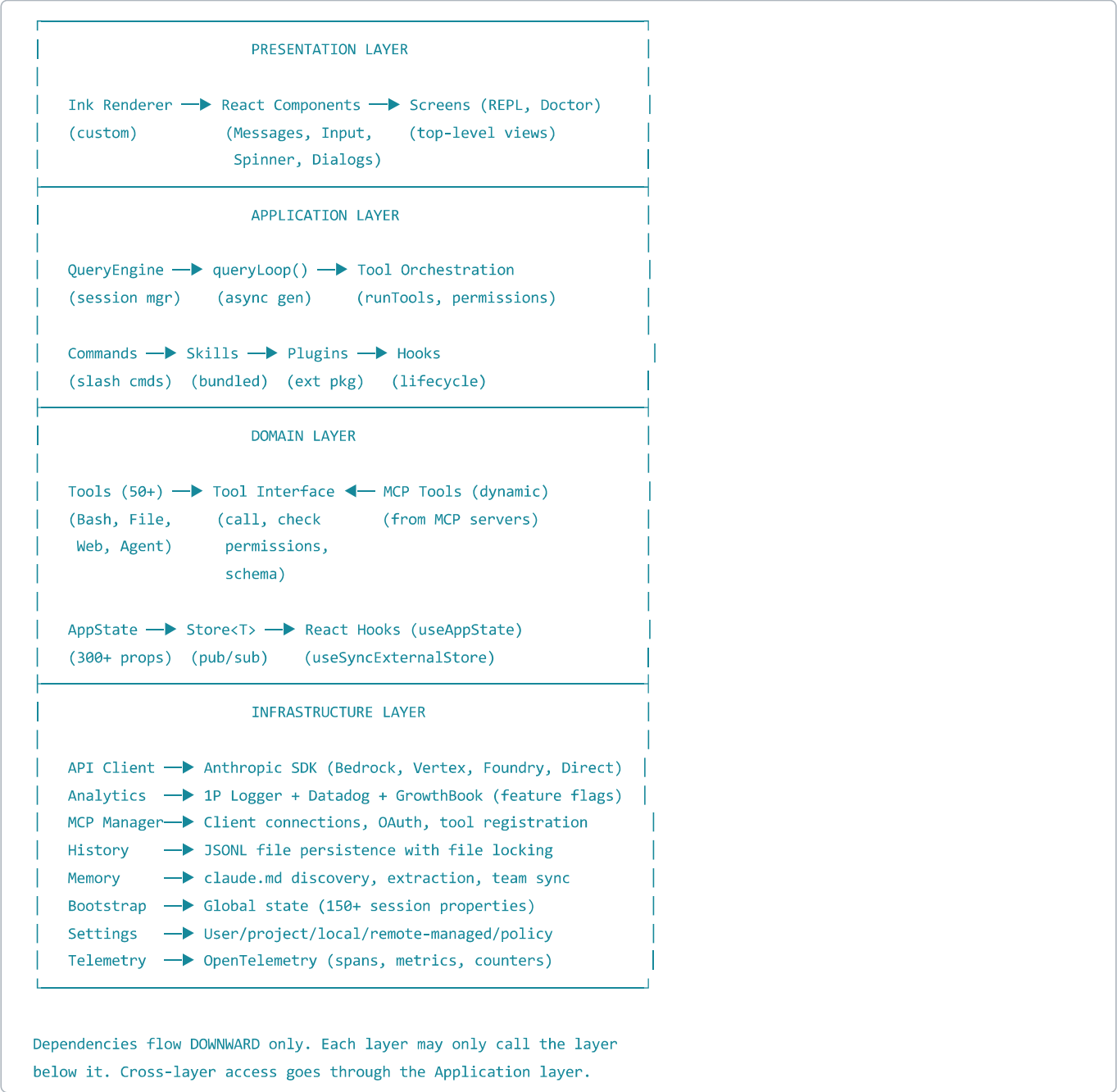
When Claude returns a tool call (e.g., **BashTool** with input `git status`), the orchestration layer:

1. **Validates** the input against the tool's Zod schema

- 2. **Checks permissions** through a multi-layer system: static rules from settings, the tool's own `checkPermissions()` logic, the current permission mode, and optionally an auto-classifier or user prompt
- 3. **Fires pre-tool hooks** (user-configured shell commands or LLM checks)
- 4. **Executes** the tool, streaming progress updates to the UI
- 5. **Fires post-tool hooks**
- 6. **Returns** the result to the query loop, which sends it back to Claude

Read-only tools (FileRead, Glob, Grep) run in **parallel batches** for speed. Write tools (FileWrite, Bash) run **sequentially** to prevent race conditions.

6. Dependency & Layer Diagram



Narrative

The architecture follows a **layered dependency model**:

- **Presentation** knows about components and screens, but not about API calls or tool execution
- **Application** orchestrates the flow: the `QueryEngine` manages conversations, commands route user intent, hooks inject lifecycle behavior
- **Domain** defines the core abstractions: what a `Tool` is, what `AppState` looks like, how permissions work
- **Infrastructure** handles I/O: API calls, file persistence, analytics, MCP connections

This separation means you can understand any layer independently. The Tool interface is the key domain abstraction — it defines a contract that 50+ implementations fulfill, and the Application layer orchestrates them without knowing their internals.

7. Key Abstractions & Data Models

The Tool Interface (src/Tool.ts)

The most important abstraction in the codebase. Every capability Claude can invoke is a Tool.

```
Tool<Input, Output, Progress>

Identity:
  name: string           "BashTool"
  userFacingName(): string "Bash"
  description(input)      "Run shell command"

Schema:
  inputSchema: ZodSchema { command: string }
  outputSchema?: ZodSchema

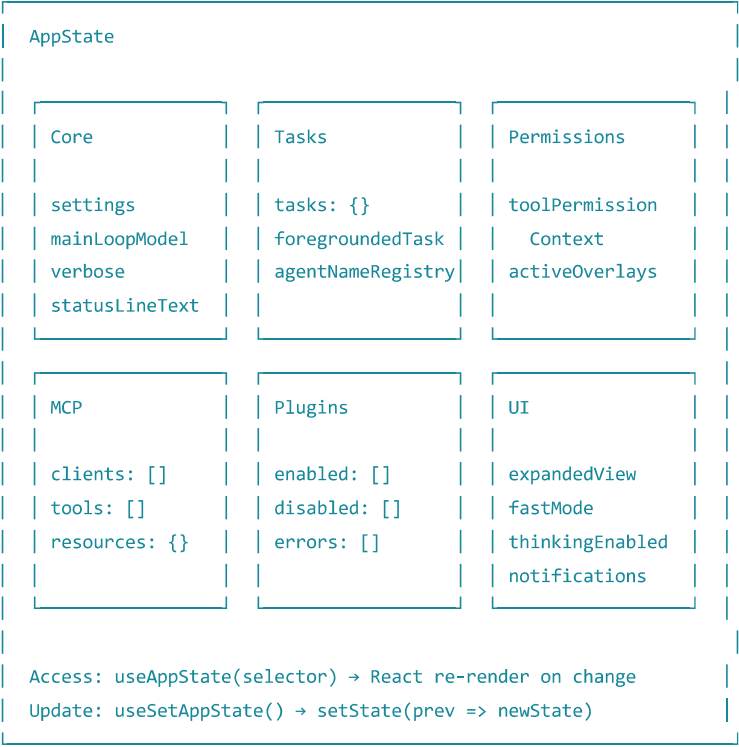
Execution:
  call(input, context)    → ToolResult
  isConcurrencySafe()     → boolean
  isReadOnly()            → boolean

Permissions:
  checkPermissions(input) → allow|deny|ask
  validateInput(input)    → ok|error

UI Rendering:
  prompt()                → system prompt text
  renderToolUseMessage()  → React component
  renderToolResultMessage() → React component
  renderToolUseProgressMessage() → React comp
```

AppState (src/state/AppStateStore.ts)

The single source of truth for all reactive UI state.



Message Types

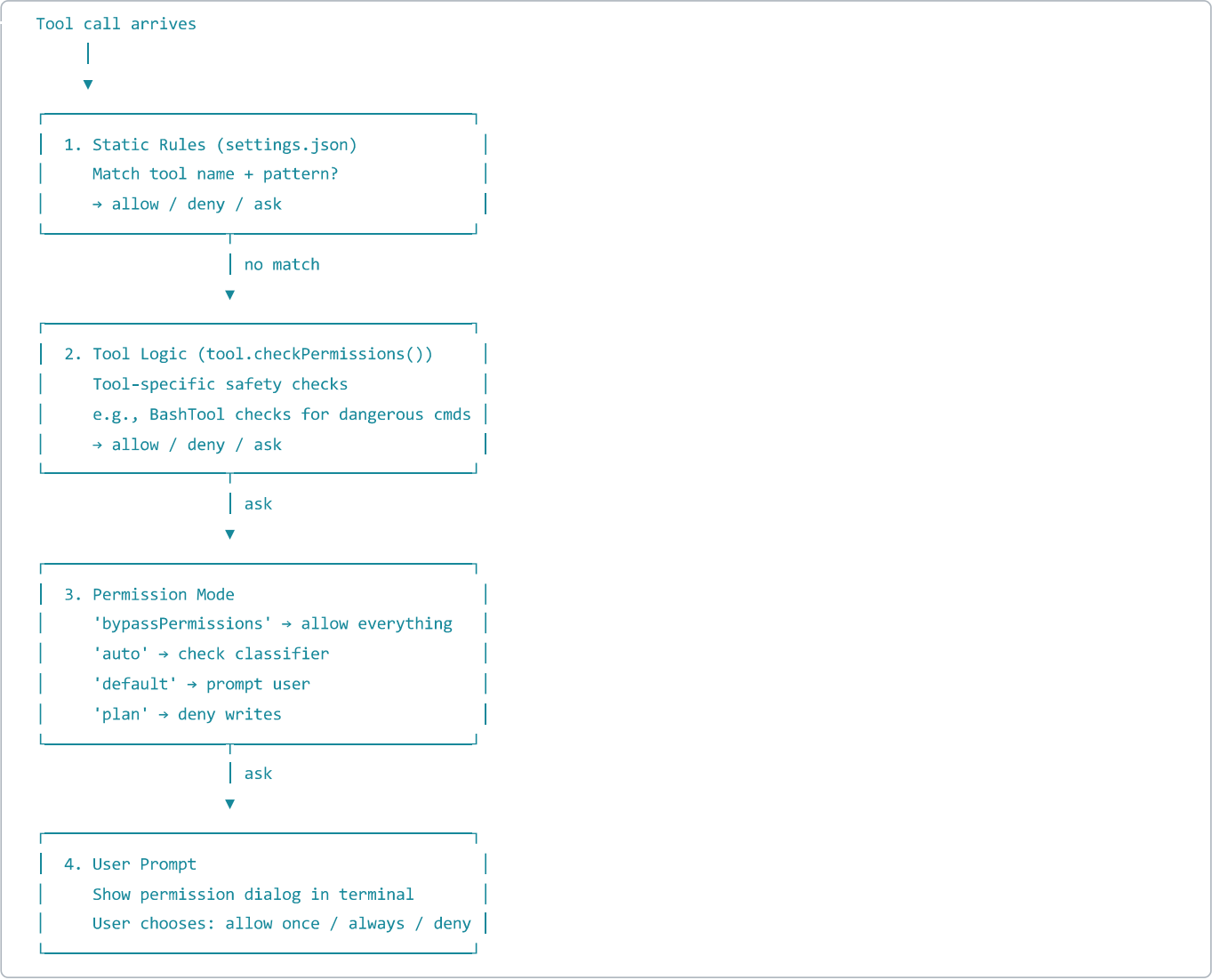
The conversation is a sequence of typed messages:

```
Message = UserMessage | AssistantMessage | SystemMessage
         | AttachmentMessage | CompactBoundaryMessage
         | ToolUseSummaryMessage | TombstoneMessage

UserMessage {
  role: 'user'
  content: TextBlock | ImageBlock | ToolResultBlock
}

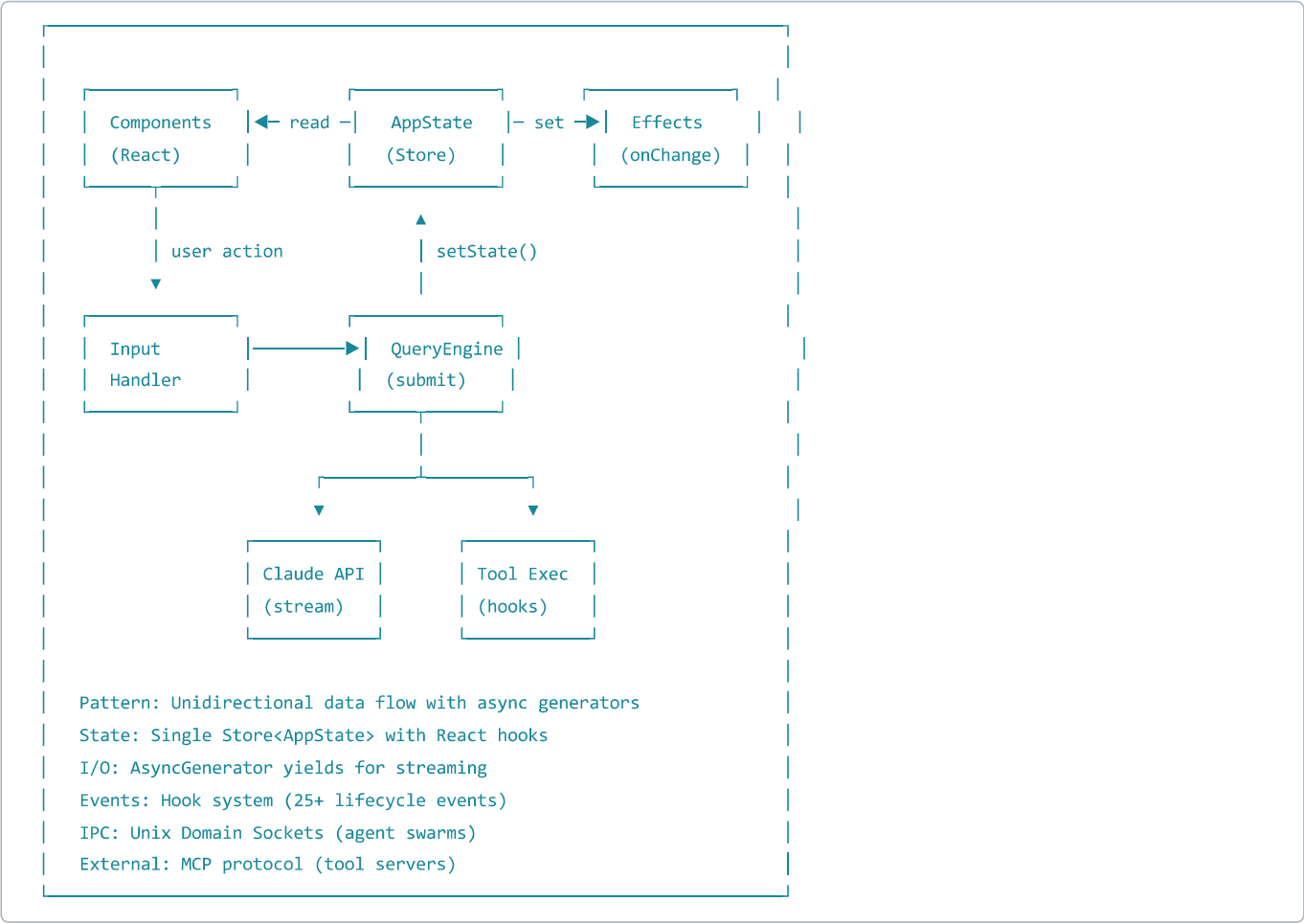
AssistantMessage {
  role: 'assistant'
  content: TextBlock | ThinkingBlock | ToolUseBlock
}
```

Permission Decision Flow

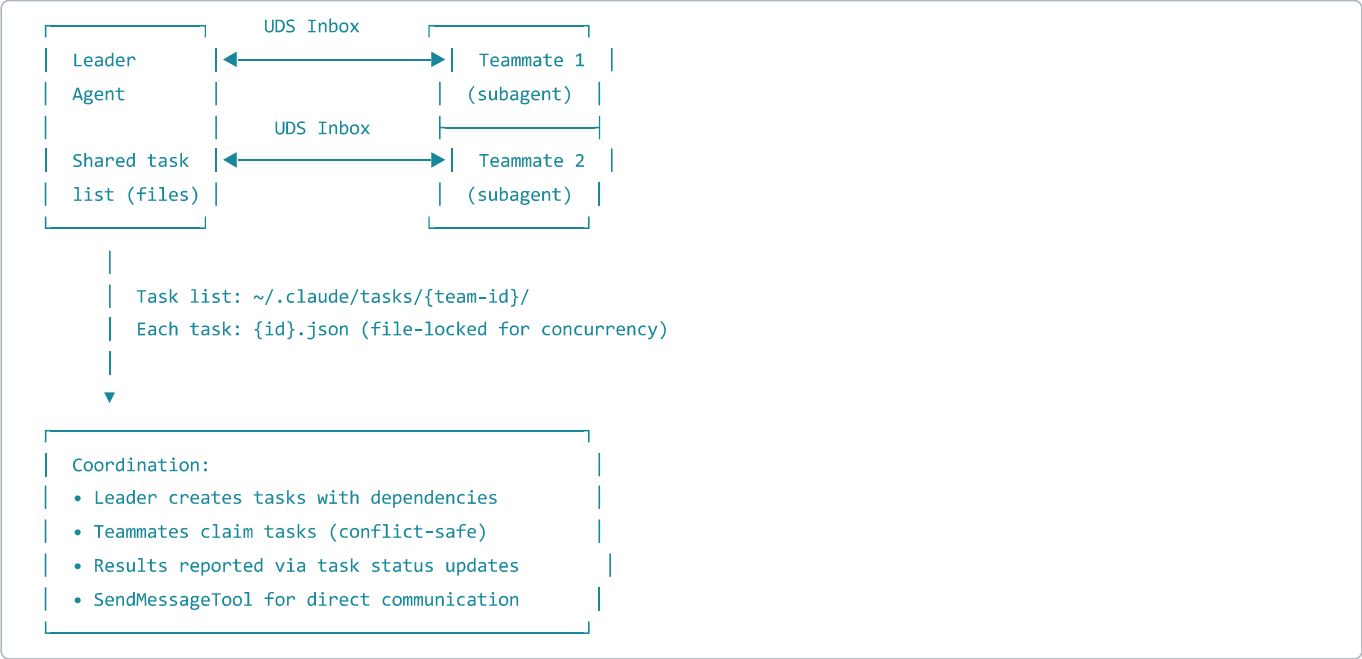


8. Communication Patterns

How Components Communicate



Inter-Agent Communication (Swarms)



9. Narrative Walkthrough: A Message End-to-End

Let's trace exactly what happens when you type "Fix the bug in auth.ts" and press Enter.

Step 1: Input Capture (Presentation Layer)

Your keystrokes flow through Ink's custom event system → `PromptInput` component → `TextInput` with history support. When you press Enter, `handlePromptSubmit()` in `utils/handlePromptSubmit.ts` is called. It checks for slash commands (none here), and passes the raw text to the REPL's message handler.

Step 2: Message Creation (Application Layer)

The REPL creates a `UserMessage` with your text as a `TextBlock`. It adds the message to the conversation history and records it to the session transcript (async, fire-and-forget for performance).

Step 3: Context Gathering (Application Layer)

Before calling the API, the system gathers context:

- **System context:** git status (branch, recent commits, dirty files)
- **User context:** contents of `CLAUDE.md` files found in the project, current date
- **System prompt:** the full system prompt including tool descriptions, permission rules, and behavioral instructions

This context is **memoized per session** — computed once, then cached.

Step 4: API Call (Infrastructure Layer)

`QueryEngine.submitMessage()` delegates to `query()` which calls `queryLoop()`. The loop calls `deps.callModel()` which maps to `queryModelWithStreaming` in `services/api/claude.ts`. This makes a streaming POST to the Anthropic Messages API with:

- The full conversation history (normalized)
- The system prompt
- All 50+ tool schemas
- Model selection and thinking configuration

Step 5: Streaming Response (Application → Presentation)

As tokens arrive from the API, the async generator `yield`s them. Each yield updates the UI in real-time — you see Claude's response character by character. The `Messages.tsx` component renders each `AssistantMessage` with `Markdown.tsx` for formatting and `HighlightedCode.tsx` for code blocks.

Step 6: Tool Calls (Application → Domain → Infrastructure)

Claude decides it needs to read the file first. Its response includes:

```
{ "type": "tool_use", "name": "Read", "input": { "file_path": "/project/auth.ts" } }
```

The query loop detects this and enters tool execution:

1. `toolOrchestration.ts` finds `FileReadTool` by name
2. `FileReadTool.validateInput()` checks the path is valid
3. `FileReadTool.checkPermissions()` — reading is typically allowed
4. Pre-tool hooks fire (if configured in settings)
5. `FileReadTool.call()` reads the file from disk
6. Post-tool hooks fire
7. The file contents are returned as a `tool_result` message

Step 7: Loop Back (Application Layer)

The tool result is added to the message history. The query loop sends the entire updated history (including the tool result) back to the Claude API. Claude now has the file contents and can analyze the bug.

Claude might then call `FileEdit` to fix the bug, triggering another round of tool execution. This time, the permission system shows a dialog because file editing requires user approval (in default mode). You see the proposed diff and press 'y' to approve.

Step 8: Completion (Application → Presentation)

After fixing the file, Claude responds with just text (no more tool calls): *"I've fixed the authentication bug in auth.ts. The issue was..."*. The query loop detects no `tool_use` blocks and returns `Terminal { reason: 'completed' }`. The UI shows Claude's final message and re-displays the input prompt.

Step 9: Persistence (Infrastructure Layer)

Throughout this flow, several things were persisted:

- **Transcript:** Full conversation saved to `~/.claude/projects/{hash}/` as JSONL
- **History:** Your input "Fix the bug in auth.ts" added to `~/.claude/history.jsonl`
- **Analytics:** Events logged (session, tool use, model, tokens, cost)
- **File state:** The read file cached in `FileStateCache` (LRU) for subsequent reads
- **Cost:** Token usage tracked in `cost-tracker.ts` and displayed in StatusLine

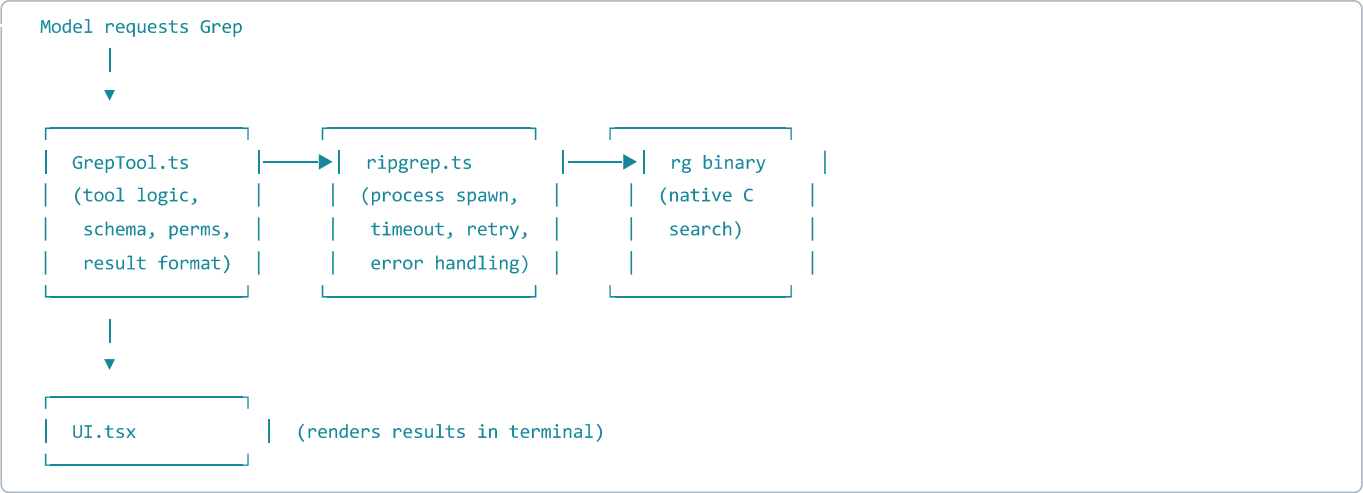
Why Is It Structured This Way?

1. **Async generators everywhere:** The streaming nature of LLM responses demands non-blocking, incremental processing. Async generators let the query loop yield events to the UI as they arrive, rather than waiting for the full response.
2. **Custom Ink renderer:** Terminal UIs need pixel-level control. The custom Ink implementation (based on React reconciliation + Yoga layout) gives Claude Code the same component model as web React, but targeting terminal cells instead of DOM nodes.
3. **Tool interface as the core abstraction:** By making every capability a Tool with a uniform interface (schema, permissions, execution), the system can add new tools without changing the query loop, permission system, or UI rendering. MCP tools from external servers plug in identically.
4. **Multi-layer permission system:** Security is critical for a code agent. The cascading permission model (rules → tool logic → mode → classifier → user) balances safety with usability — safe operations auto-approve, dangerous ones require explicit consent.
5. **Feature flags via compile-time elimination:** Bun's `feature()` intrinsic lets Anthropic ship a single binary with 108+ internal modules completely stripped out. The published CLI contains zero internal code — it's all dead-code-eliminated at build time.
6. **Bootstrap state as global mutable singleton:** While controversial, this pattern enables fast startup (no DI container) and easy access from any module. The `bootstrap/state.ts` file is the "session object" that everything reads from.
7. **Fire-and-forget parallelism:** Startup parallelizes aggressively — network pre-connection, auth prefetch, repository detection, and command loading all run concurrently. This is why Claude Code starts fast despite doing a lot of initialization.

10. Deep Dive: The Grep Tool (Agentic Search)

The Grep tool is one of the most frequently invoked tools in the agentic loop. It's a read-only, concurrency-safe wrapper around `ripgrep` (`rg`) that the model uses to search codebases.

Architecture



Key files:

- `src/tools/GrepTool/GrepTool.ts` — Tool definition, input schema, execution logic, result formatting
- `src/tools/GrepTool/prompt.ts` — Tool description injected into Claude's system prompt
- `src/tools/GrepTool/UI.tsx` — Terminal rendering of search results
- `src/utils/ripgrep.ts` — Low-level ripgrep binary management, process spawning, error recovery

Input Schema

Defined via Zod with the following parameters:

Parameter	Purpose
<code>pattern</code>	Regex pattern (ripgrep syntax, not POSIX grep)
<code>path</code>	Directory or file to search (defaults to CWD)
<code>glob</code>	File filter like <code>"*.ts"</code> or <code>"*.{js,tsx}"</code>
<code>type</code>	Ripgrep file type shorthand (<code>js</code> , <code>py</code> , <code>rust</code>)
<code>output_mode</code>	<code>files_with_matches</code> (default), <code>content</code> , or <code>count</code>
<code>-A/-B/-C/context</code>	Context lines around matches (content mode only)
<code>-n</code>	Line numbers (defaults <code>true</code> , content mode only)
<code>-i</code>	Case-insensitive search
<code>multiline</code>	Enable cross-line matching (<code>-U --multiline-dotall</code>)
<code>head_limit</code>	Cap results (default <code>250</code> , <code>0</code> = unlimited)
<code>offset</code>	Skip first N entries (for pagination)

The schema uses `semanticNumber` and `semanticBoolean` wrappers that handle the model sending strings like `"true"` or `"3"` instead of actual booleans/numbers.

Execution Flow

```
GrepTool.call()
├─ 1. BUILD RIPGREP ARGS
│  └─ Always: --hidden, --max-columns 500
│  └─ Auto-exclude VCS dirs: .git, .svn, .hg, .bzz, .jj, .sl
│  └─ Map output_mode → rg flags (-l, -c, or default)
│  └─ If pattern starts with "-" → use -e flag
│  └─ Apply glob filters (split on spaces, preserve brace patterns)
│  └─ Apply permission-based ignore patterns (e.g. node_modules, .env)
│  └─ Exclude orphaned plugin cache directories
├─ 2. SPAWN RIPGREP (ripgrep.ts)
│  └─ Resolve rg binary (system → embedded → builtin vendor)
│  └─ Timeout: 20s (60s on WSL), configurable via env var
│  └─ Buffer: 20MB max stdout
│  └─ On EAGAIN error → retry with -j 1 (single-threaded)
│  └─ On timeout with no output → throw RipgrepTimeoutError
│  └─ On macOS → auto-codesign vendored binary if needed
├─ 3. PROCESS RESULTS (by output_mode)
│  └─ files_with_matches (default):
│     └─ stat() each file (Promise.allSettled, tolerates deletions)
│     └─ Sort by modification time (most recent first)
│     └─ Apply head_limit/offset pagination
│     └─ Convert to relative paths
│  └─ content:
│     └─ Apply head_limit/offset first (avoid wasted path conversion)
│     └─ Convert absolute paths in "path:line:content" to relative
│  └─ count:
│     └─ Apply head_limit/offset
│     └─ Convert paths to relative
│     └─ Parse "path:count" lines → total matches + file count
```

Ripgrep Binary Resolution (`src/utils/ripgrep.ts`)

The `getRipgrepConfig()` function (memoized, resolved once per process) picks the `rg` binary in priority order:

1. **System** — user's installed `rg` (used if `USE_BUILTIN_RIPGREP` is not set to falsy). Uses just the command name `rg` rather than the resolved path to prevent PATH hijacking.
2. **Embedded** — in the production Bun build, ripgrep is statically compiled into the binary. Spawned via `argv0='rg'` so the Bun process dispatches to the embedded `rg`.
3. **Builtin** — vendored platform-specific binary from `vendor/ripgrep/{arch}-{platform}/rg`.

Error Recovery

The ripgrep wrapper has several resilience mechanisms:

- **EAGAIN retry**: Resource-constrained environments (Docker, CI) can fail with “resource temporarily unavailable” when `rg` spawns too many threads. The wrapper detects this and retries with `-j 1` (single-threaded) for that call only — it doesn't persist single-threaded mode globally, which would cause timeouts on large repos.
- **Timeout with partial results**: If `rg` times out but has partial stdout, the last line is dropped (may be incomplete) and the partial results are returned. If there's no output at all, a `RipgrepTimeoutError` is thrown so the model knows the search didn't complete (vs. thinking there were no matches).
- **Buffer overflow**: At 20MB stdout, partial results are returned with the last line dropped.
- **File deletion races**: `files_with_matches` mode uses `Promise.allSettled` for stat calls, so a file deleted between `rg`'s scan and the stat doesn't crash the whole search — it just gets `mtime: 0`.

Pagination System

The default `head_limit` of 250 prevents context bloat — unbounded content-mode greps can fill the 20KB tool-result persistence threshold (6–24K tokens). When truncation occurs, `appliedLimit` is set in the output:

```
[Showing results with pagination = limit: 250]
```

This tells the model “there are more results.” It can then call Grep again with `offset: 250` to see the next page. This is how the model implements **multi-round search** — paginating through large result sets across multiple tool calls.

Result Formatting

`mapToolResultToToolResultBlockParam()` formats output into text `tool_result` blocks sent back to Claude:

- `files_with_matches`: "Found 5 files\nsrc/a.ts\nsrc/b.ts\n..." — files sorted by mtime (most recently modified first, a heuristic that the most task-relevant files were touched recently)
- `content`: Matching lines with relative paths and line numbers
- `count`: "src/a.ts:12\nsrc/b.ts:3\n\nFound 15 total occurrences across 2 files."

If results exceed `maxResultSizeChars` (20,000 chars), the tool system persists them to disk and sends Claude a preview + filepath instead, preventing context window overflow.

Permission & Security

- `validateInput()` checks the path exists, suggests corrections if not found, and **blocks UNC paths** (`\\server\share`) to prevent NTLM credential leaks on Windows
- `checkPermissions()` delegates to `checkReadPermissionForTool()` — standard read-permission check against `toolPermissionContext`
- `preparePermissionMatcher()` enables wildcard permission rules like `Grep(*.env)` to deny searching sensitive file patterns
- Permission-based ignore patterns from settings (e.g., `node_modules`, `.env` files) are automatically applied as `--glob !pattern` arguments
- The tool's prompt explicitly instructs the model to **never** run `grep` or `rg` via BashTool — always use this tool, which enforces the correct permission and ignore-pattern rules

Tool Properties

`isConcurrencySafe: true` ← multiple Greps run in parallel batches

`isReadOnly: true` ← no filesystem mutations

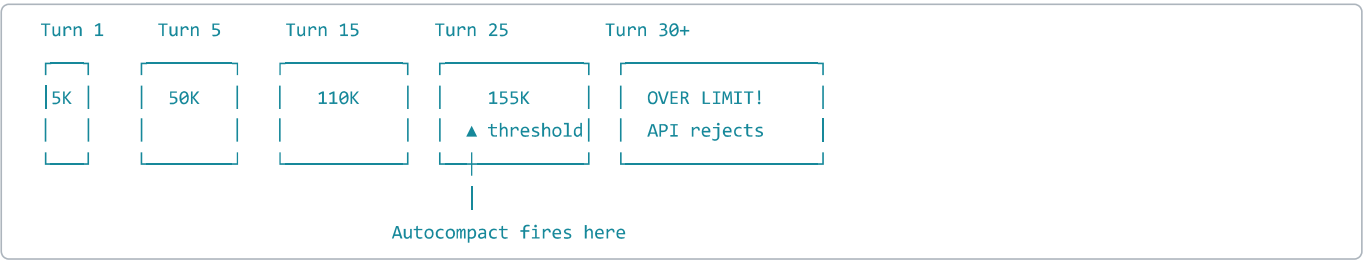
`maxResultSizeChars: 20,000` ← results >20K persisted to disk

`strict: true` ← no extra input fields allowed

11. Deep Dive: Memory Management & Context Compaction

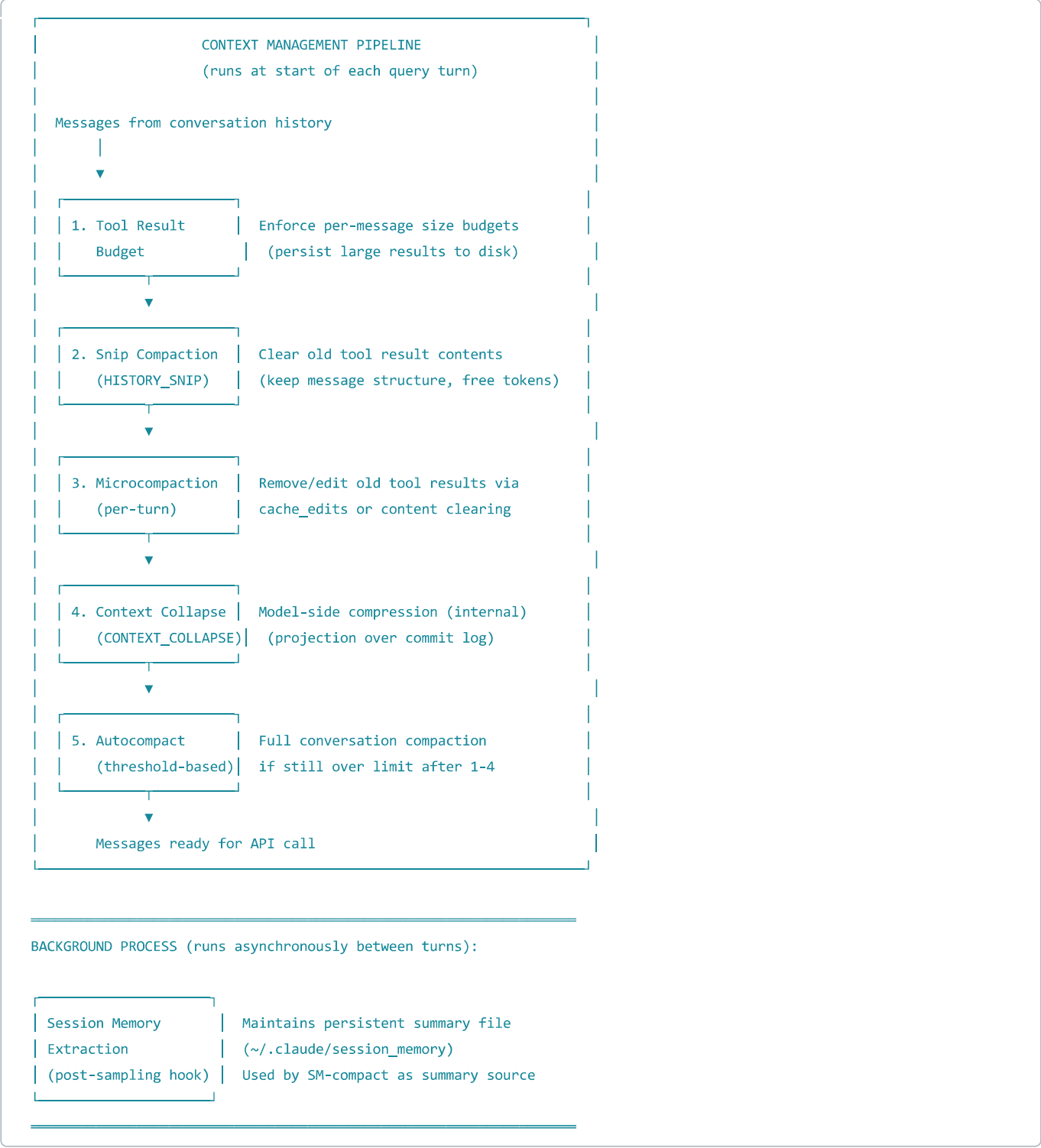
Claude Code operates within a finite context window (e.g., ~200K tokens). Conversations with heavy tool use can fill this quickly. The system uses a **multi-layered compaction strategy** that operates at three timescales to keep conversations going indefinitely.

The Problem



Without compaction, long sessions would hit the context limit and stop working. The system must compress old context while preserving the information the model needs to continue working effectively.

Architecture Overview



Key Files

File	Purpose
src/services/compact/autoCompact.ts	Autocompact threshold logic, orchestration
src/services/compact/compact.ts	Full conversation compaction (forked agent)
src/services/compact/sessionMemoryCompact.ts	Session-memory-based compaction (lightweight)
src/services/compact/microCompact.ts	Per-turn tool result clearing
src/services/compact/apiMicrocompact.ts	API-native context management
src/services/compact/postCompactCleanup.ts	Cache invalidation after compaction
src/services/SessionMemory/sessionMemory.ts	Background session memory extraction

File	Purpose
<code>src/services/tokenEstimation.ts</code>	Token counting (API + heuristic)
<code>src/utils/tokens.ts</code>	Canonical token count function
<code>src/query.ts</code>	Integration point — runs the pipeline each turn

How Tokens Are Counted

The system uses **hybrid token counting** (`src/services/tokenEstimation.ts`):

- **API-based:** `anthropic.beta.messages.countTokens()` for precise counts
- **Heuristic fallback:** ~1 token per 4 characters (2 chars/token for JSON, fixed 2000 tokens for images)
- **Canonical function** (`tokenCountWithEstimation()` in `src/utils/tokens.ts`): Finds the last API response with usage data, walks backward to include all interleaved `tool_results`, then adds rough estimates for messages added since. Returns: **API usage** (`input + output + cache`) + **estimated new messages**

The effective context window is calculated as:

```
effectiveWindow = modelContextWindow - reservedForSummary(20,000 tokens)
```

The 20K reserve ensures the compaction API call itself doesn't hit "prompt too long."

Layer 1: Tool Result Budget

Before any compaction runs, `applyToolResultBudget()` enforces per-message size limits. Tool results exceeding `maxResultSizeChars` (e.g., 20K chars for Grep) are persisted to disk and replaced with a preview + filepath. This prevents individual messages from consuming disproportionate context.

Layer 2: Snip Compaction (feature-gated: **HISTORY_SNIP**)

The lightest compaction. Clears old tool result `contents` to `[Old tool result content cleared]` while keeping the message structure intact (`tool_use/tool_result` pairing preserved). No API call needed. The tokens freed are tracked and passed to autocompact so its threshold calculation accounts for the savings.

Layer 3: Microcompaction (per-turn)

Removes old tool results without invalidating the prompt cache. Two mechanisms:

Cached Microcompaction (primary, feature-gated: `CACHED_MICROCOMPACT`): Uses the `cache_edits` API to delete old tool results while keeping the prompt prefix cached. No message content is modified — edits are piggybacked on the next API request. The system tracks compactable tools (Bash, Glob, Grep, WebSearch, WebFetch, FileEdit, FileWrite, FileRead) and keeps only the N most recent results.

Time-Based Microcompaction (fallback): If the gap since the last assistant message exceeds a threshold (~1 hour, meaning the prompt cache has expired), content-clears old tool results directly. Since the cache is cold anyway, mutating messages is safe.

Layer 4: Context Collapse (feature-gated: **CONTEXT_COLLAPSE**, internal)

An alternative model-side compression system. Projects a collapsed view over a commit log of conversation segments. When enabled, it suppresses proactive autocompact (they'd race and corrupt each other). Operates with its own thresholds: 90% commit point, 95% blocking-spawn headroom.

Layer 5: Autocompact (threshold-based)

The main compaction mechanism. Triggered when tokens reach `effectiveContextWindow - 13,000`.

```
shouldAutoCompact() decision:
|
|— Is autocompact enabled? (not disabled via env/config)
|— Token count >= threshold? (effectiveWindow - 13K)
|— Not a forked agent? (session_memory, compact agents excluded)
|— Not in reactive-only mode? (REACTIVE_COMPACT gate)
|— Not in context-collapse mode? (CONTEXT_COLLAPSE gate)
|
|   ▼ All true → attempt compaction
```

Autocompact has two strategies, tried in order:

Strategy A: Session Memory Compaction (SM-Compact)

The lightweight path. Uses the session memory summary file (maintained in background) to replace old messages without an API call.

```
trySessionMemoryCompaction()
|
|— 1. Wait for in-flight session memory extraction to finish
|— 2. Load session memory summary from disk
|— 3. Find lastSummarizedMessageId (boundary marker)
|— 4. Calculate which messages to keep:
|   |— Preserve messages after summarization boundary
|   |— Expand backward to meet minimums:
|   |   min 10K tokens, min 5 text-block messages
|   |— Cap at 40K tokens max
|   |— Adjust for API invariants:
|   |   |— Never split tool_use/tool_result pairs
|   |   |— Never orphan thinking blocks (same message.id)
|   |   |— Never drop below last compact_boundary_message
|   |
|— 5. Build post-compact messages:
|   [CompactBoundary] + [SessionMemorySummary] + [KeptRecentMessages]
|
|— 6. Estimate post-compact token count
|   |— If still >= threshold → abort (not effective enough)
|
|— 7. Return CompactionResult
```

Strategy B: Full Conversation Compaction (fallback)

If SM-compact isn't available or wasn't effective, runs a **forked agent** to summarize the entire conversation.

```
compactConversation()
|
|— 1. Strip images (replace with [image] placeholders)
|— 2. Group messages by API round
|— 3. Send to forked compaction agent with structured prompt:
|   |— Primary Request and Intent
|   |— Key Technical Concepts
|   |— Files and Code Sections (with snippets)
|   |— Errors and Fixes
|   |— Problem Solving
|   |— All User Messages (verbatim)
|   |— Pending Tasks
|   |— Current Work
|   |— Optional Next Step
|
|— 4. Agent returns <analysis> (scratchpad) + <summary> (output)
|— 5. Build post-compact messages:
|   [CompactBoundary] + [Summary] + [Attachments] + [KeptMessages]
|
|— 6. Post-compact attachments:
|   |— Restore up to 5 recently-used files (50K token budget)
|   |— Re-inject invoked skills (up to 5, 25K budget)
|   |— Re-inject plan mode state (if active)
|   |— Notify about running async agents
```

Circuit Breaker: After 3 consecutive autocompact failures, the system stops retrying to avoid API spam.

Background: Session Memory Extraction

Running asynchronously via a post-sampling hook, the Session Memory service maintains a persistent summary file at `~/ .claude/projects/<path>/ .claude/session_memory`.

Trigger conditions (all must be true):

- First extraction: total context $\geq$ 8,000 tokens
- Subsequent extractions: context grown $\geq$ 15,000 tokens since last extraction AND ($\geq$ 5 tool calls OR natural break point)

Process: A forked agent updates structured sections of the session memory file using the Edit tool:

```
# Session Title
# Current State      ← what's being worked on now
# Task specification  ← original user request
# Files and Functions ← important files and contents
# Workflow           ← commands, execution order
# Errors & Corrections ← failed approaches, fixes
# Codebase Documentation ← how components fit together
# Learnings          ← what worked, what to avoid
# Key results        ← outputs user requested
# Worklog            ← step-by-step summary
```

Each section capped at ~2,000 tokens, total file at ~12,000 tokens. The extraction coalesces rapid calls — if an extraction is in progress, the next one queues and runs after.

Post-Compact Cleanup

After any compaction, `runPostCompactCleanup()` invalidates stale caches:

- Microcompact state (tool registrations become invalid)
- Context collapse state
- Memory file cache (CLAUDE.md files might have changed)
- Session message cache
- Classifier approvals (file permissions might change)

- Speculative shell checks
- Beta tracing state

Only main-thread state is cleared for main-thread compactions (subagents share module-level state).

Integration in the Query Loop (src/query.ts)

Every turn, the pipeline runs in this exact order at lines 365–468:

```
1. getMessagesAfterCompactBoundary()    ← start from last compact point
2. applyToolResultBudget()              ← enforce per-message size limits
3. snipCompactIfNeeded()                ← clear old tool contents (HISTORY_SNIP)
4. microcompactMessages()               ← remove old tool results
5. contextCollapse.applyCollapsesIfNeeded() ← model-side compression
6. autoCompactIfNeeded()                ← full compaction if still over
7. → messages ready for API call
```

Each layer reduces the token count. If an earlier layer brings the count below the autocompact threshold, later layers become no-ops.

Reactive vs. Proactive Compaction

Proactive (default): Compact before hitting the API limit. The 13K-token buffer gives headroom to compaction completes before the API rejects requests.

Reactive (feature-gated: `REACTIVE_COMPACT`): Only compact after the API returns a “prompt too long” error (HTTP 413). When enabled, suppresses proactive autocompact. The query loop withholds 413 errors, runs compaction, then retries.

Lifecycle of a Long Conversation

```
Turn 1:   ~5K tokens    No compaction needed
Turn 5:   ~50K tokens   Session memory extraction triggers (first time)
Turn 10:  ~90K tokens   Microcompact clearing old tool results each turn
Turn 15:  ~110K tokens  Session memory updates again (15K new tokens)
Turn 20:  ~135K tokens  Snip clears more old tool contents
Turn 25:  ~155K tokens  ══ AUTOCOMPACT TRIGGERS ══
                |
                ├── Try SM-compact → uses session memory summary
                |   └─ If effective: messages = [summary] + [recent ~40K]
                ├── Else: full compaction via forked agent
                |   └─ messages = [summary] + [attachments] + [recent tail]
                ├── Post-compact cleanup (invalidate caches)
                └─ Result: ~60-80K tokens (fresh start)

Turn 26+: Resume from compact boundary, cycle continues
          Session memory keeps tracking NEW work
          If context fills again → another autocompact cycle
```

Key Constants

Constant	Value	Purpose
<code>AUTOCOMPACT_BUFFER_TOKENS</code>	13,000	Trigger: <code>effectiveWindow</code> - 13K
<code>WARNING_THRESHOLD_BUFFER_TOKENS</code>	20,000	User warning zone
<code>MAX_OUTPUT_TOKENS_FOR_SUMMARY</code>	20,000	Reserved for compaction API call
<code>MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES</code>	3	Circuit breaker
SM-compact <code>minTokens</code>	10,000	Minimum tokens to preserve
SM-compact <code>maxTokens</code>	40,000	Maximum tokens to preserve

Constant	Value	Purpose
SM-compact <code>minTextBlockMessages</code>	5	Minimum messages with text
Session memory init threshold	8,000	First extraction trigger
Session memory update interval	15,000	Subsequent extraction gap
Session memory section max	~2,000	Per-section token limit
Session memory total max	~12,000	Total summary file limit
Post-compact file budget	50,000	Token budget for file restoration
Post-compact skill budget	25,000	Token budget for skill restoration